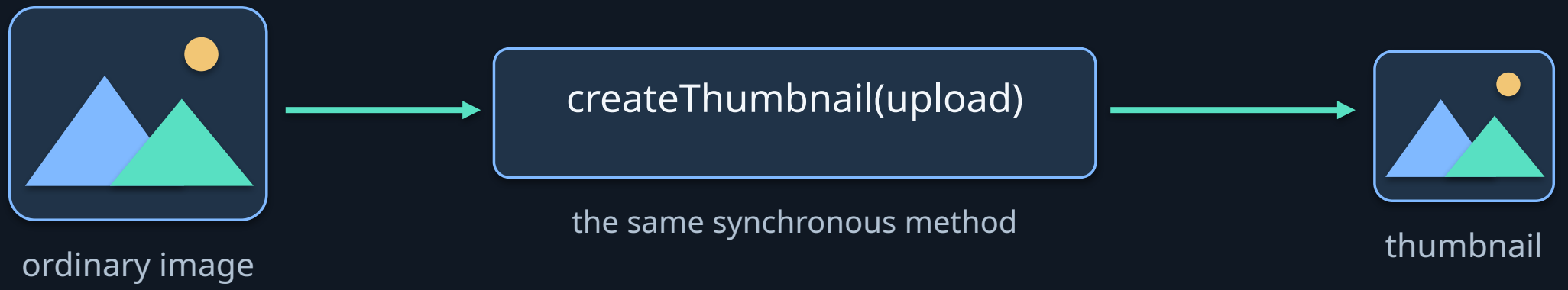
SIMPLIFIED EXAMPLE

One JVM. Different permissions.



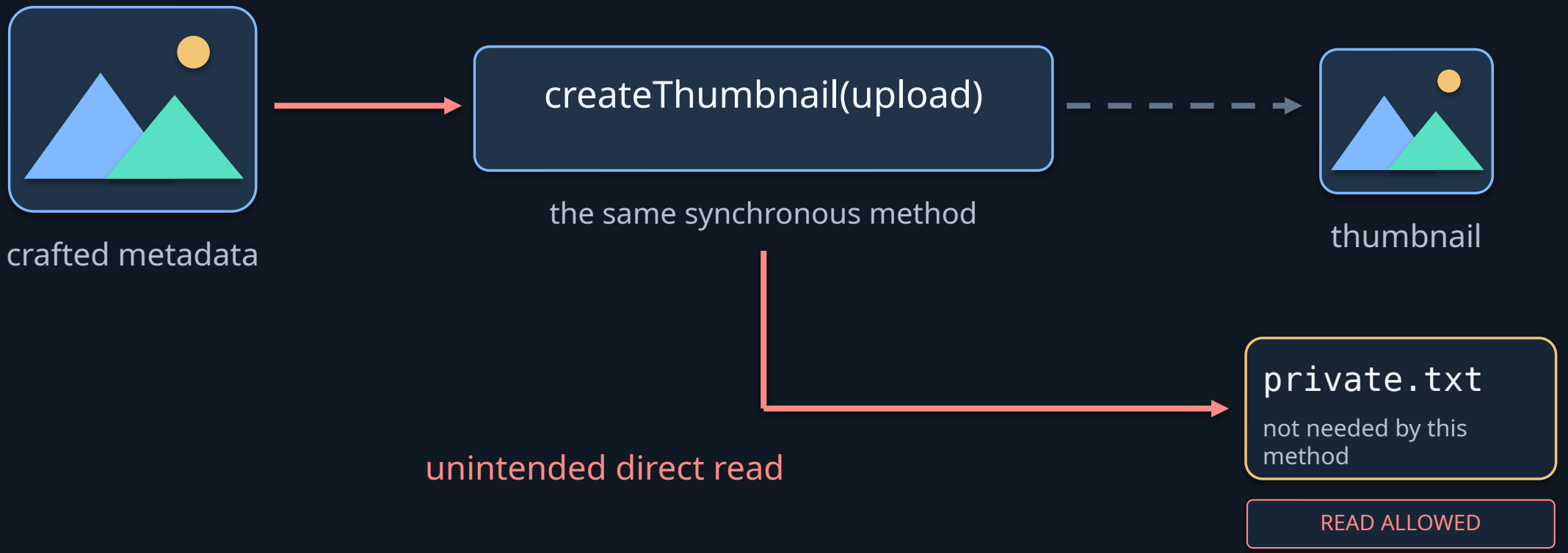
Why should this method be able to open that file?

`private.txt`

not needed by this method

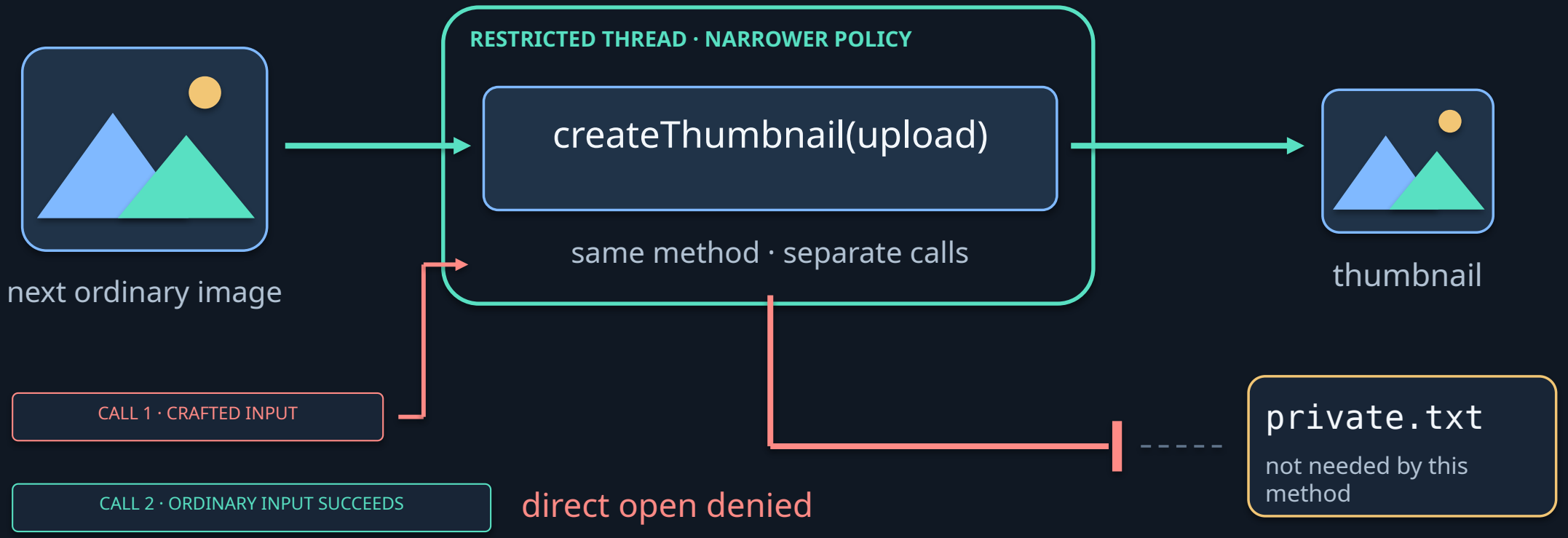
Leanid Piliptsevich · Linux kernel-enforced restrictions for selected JVM work

The same method can take an unintended path.



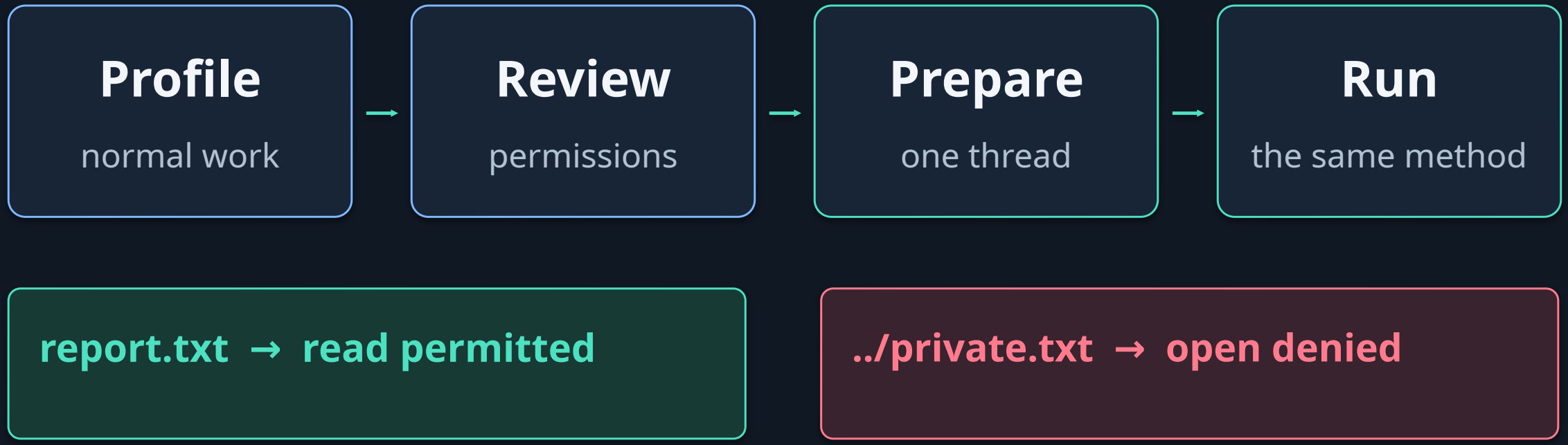
Same method. Crafted input. One unwanted synchronous read.

Deny the extra action. Keep the useful work.



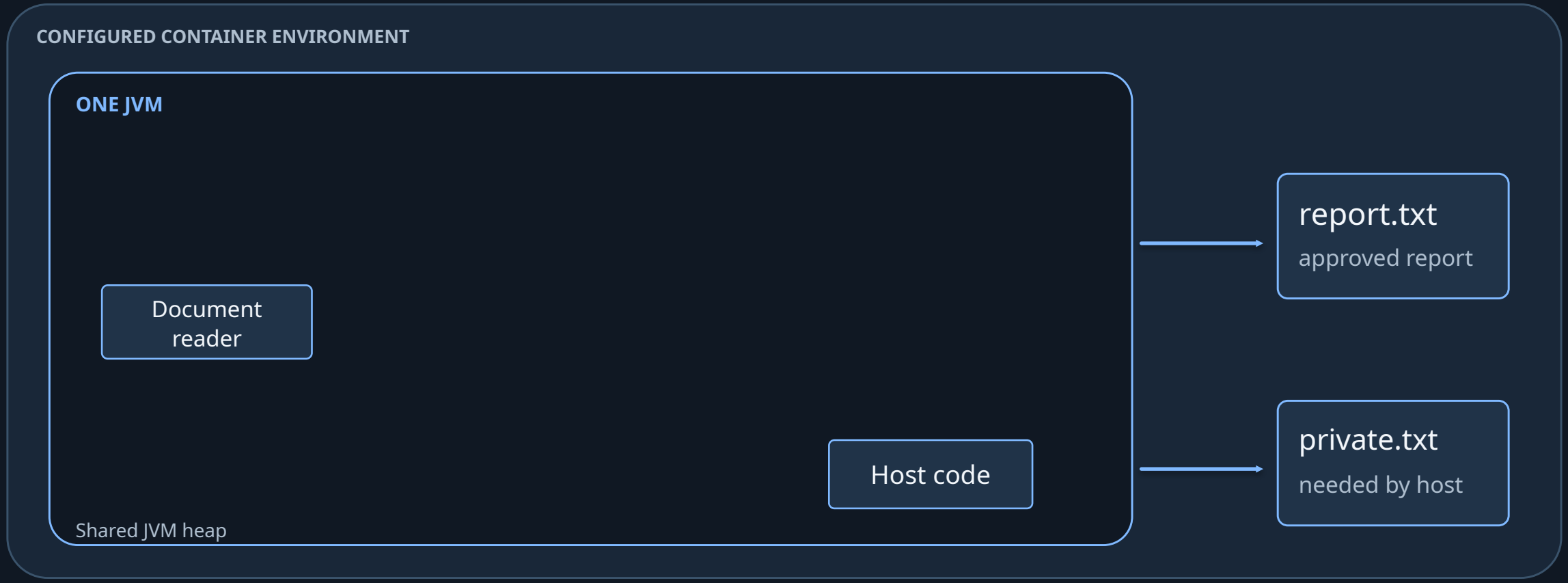
Narrow the direct OS permissions of this execution.

Profile normal work. Restrict later calls.



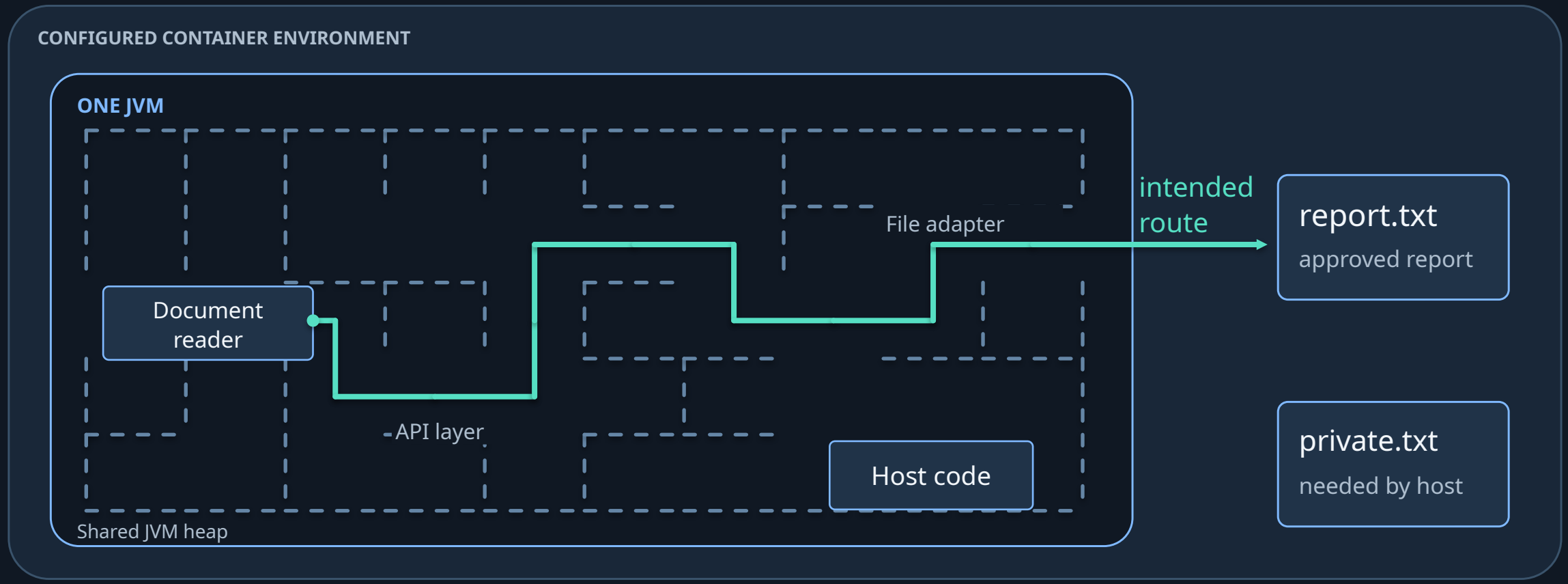
Same code. A reviewed policy. No attack detector.

Your container is still doing its job.



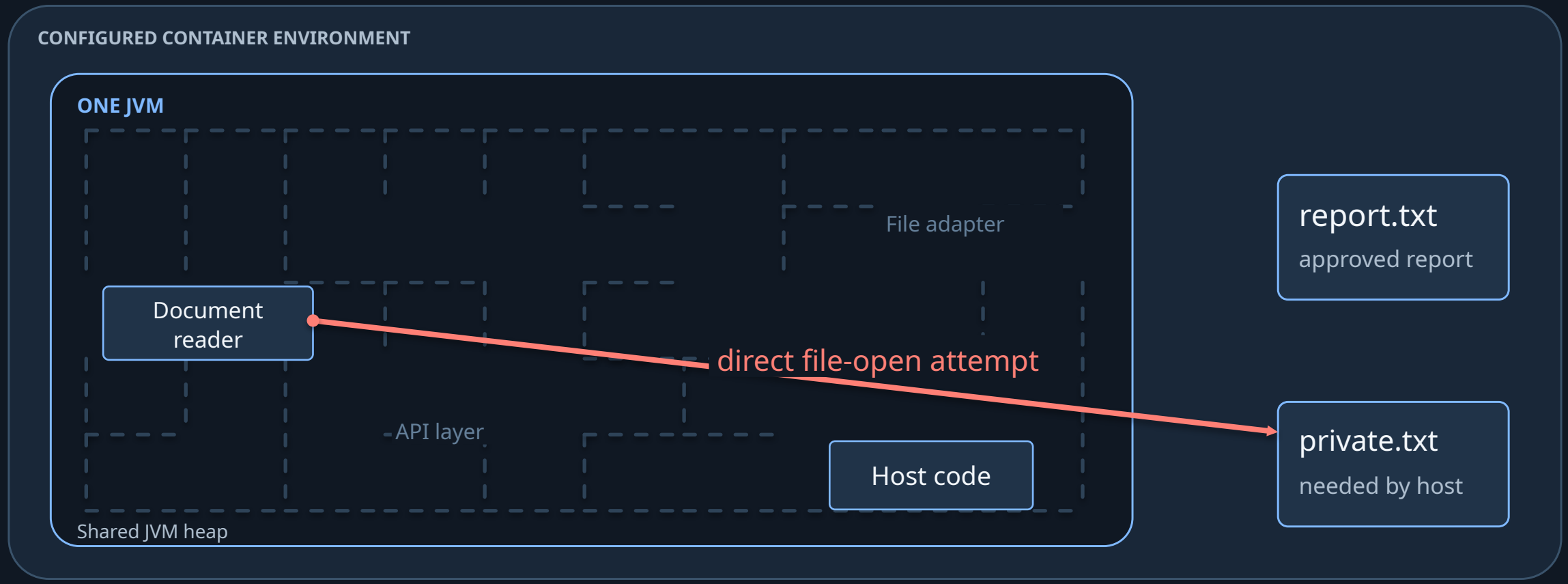
The application needs both resources. This operation does not.

Developers see a maze of intended calls.



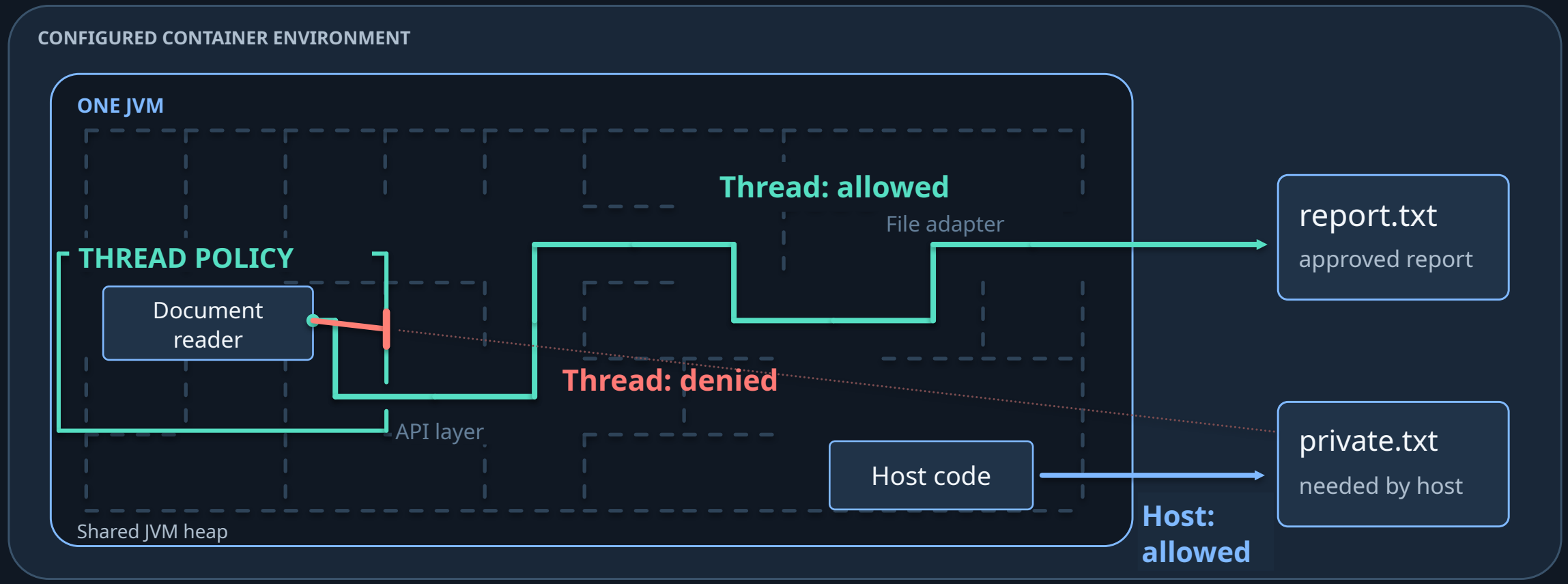
Dashed walls: intended calls. Green route: the normal path to the input.

The OS does not see those walls.



The logical walls fade. The configured outer restrictions remain.

Give selected work its own OS policy.



The policy controls access—not the route through the maze.

Profile the operation with normal input.

1 PROFILE | 2 REVIEW + PREPARE | 3 NORMAL CALL | 4 ATTACKER CALL

DEVELOPMENT / TEST · WALKTHROUGH PSEUDOCODE

```
observed = profile {
    readDocument("report.txt")
}
```

FILE ACCESS IN THE PROFILE

READ
`/demo/docs/report.txt`

The profile also includes system calls and JVM activity.

Profile normal work first. Decide permissions separately.

Review the profile. Prepare the thread.

1 PROFILE | 2 REVIEW + PREPARE | 3 NORMAL CALL | 4 ATTACKER CALL

WALKTHROUGH PSEUDOCODE

```
candidate = policyFrom(observed)
policy = review(candidate)
thread = prepareRestrictedThread(policy)
```

review = a developer decision

REVIEWED FILE-ACCESS POLICY

ALLOW READ

`/demo/docs/report.txt`

NOT GRANTED

`/demo/private.txt`

Required JVM support retained

Profile in development. Enforce a fixed, reviewed policy at runtime.

Normal input still works.

1 PROFILE | 2 REVIEW + PREPARE | 3 NORMAL CALL | 4 ATTACKER CALL

RUNTIME · PREPARED RESTRICTED THREAD

```
executeOn(thread) {
    readDocument("report.txt")
}
```

READ PERMITTED

`/demo/docs/report.txt`

Return report contents

Same method. Reviewed policy installed. Profiler not running.

The operation still has the permission its normal input needs.

Attacker input reaches the bug—not the file.

1 PROFILE | 2 REVIEW + PREPARE | 3 NORMAL CALL | 4 ATTACKER CALL

RUNTIME · SAME THREAD · SAME POLICY

```
executeOn(thread) {
    readDocument("../private.txt")
}
```

OPEN DENIED

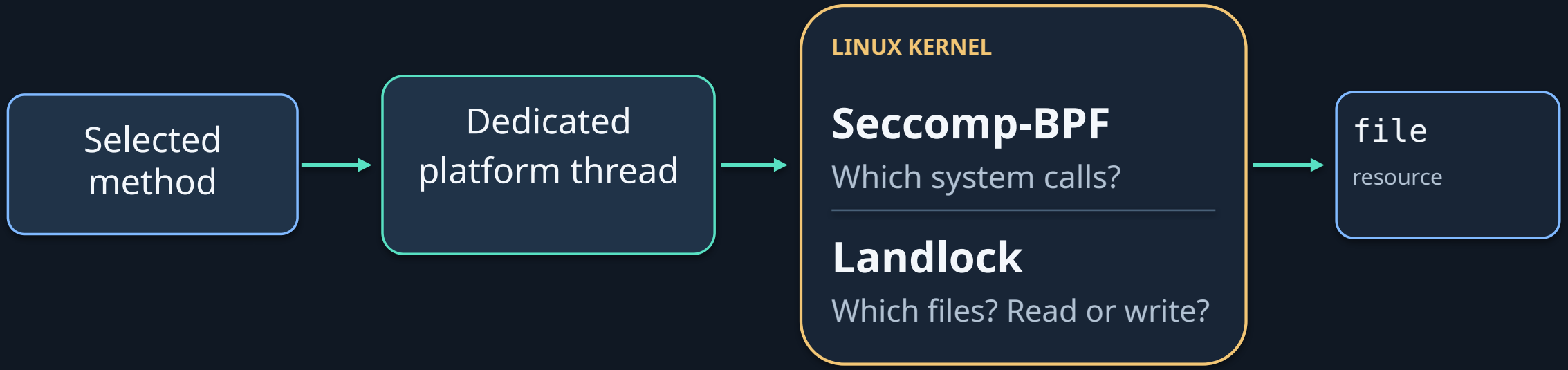
`/demo/private.txt`

Private contents not returned

Handle the error → next readDocument("report.txt") succeeds

The policy blocks the file access—not a particular spelling of the input.

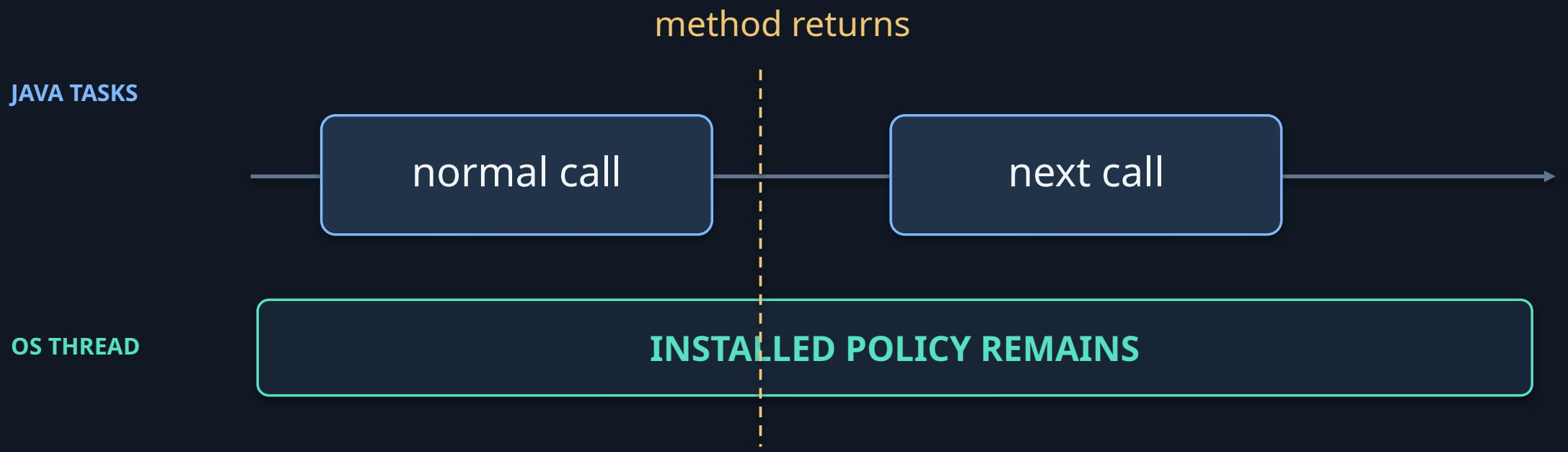
Linux checks system calls and file access.



MazeWall chooses the thread.
Linux checks the operation.

In this example: seccomp filters syscalls; Landlock controls file access.

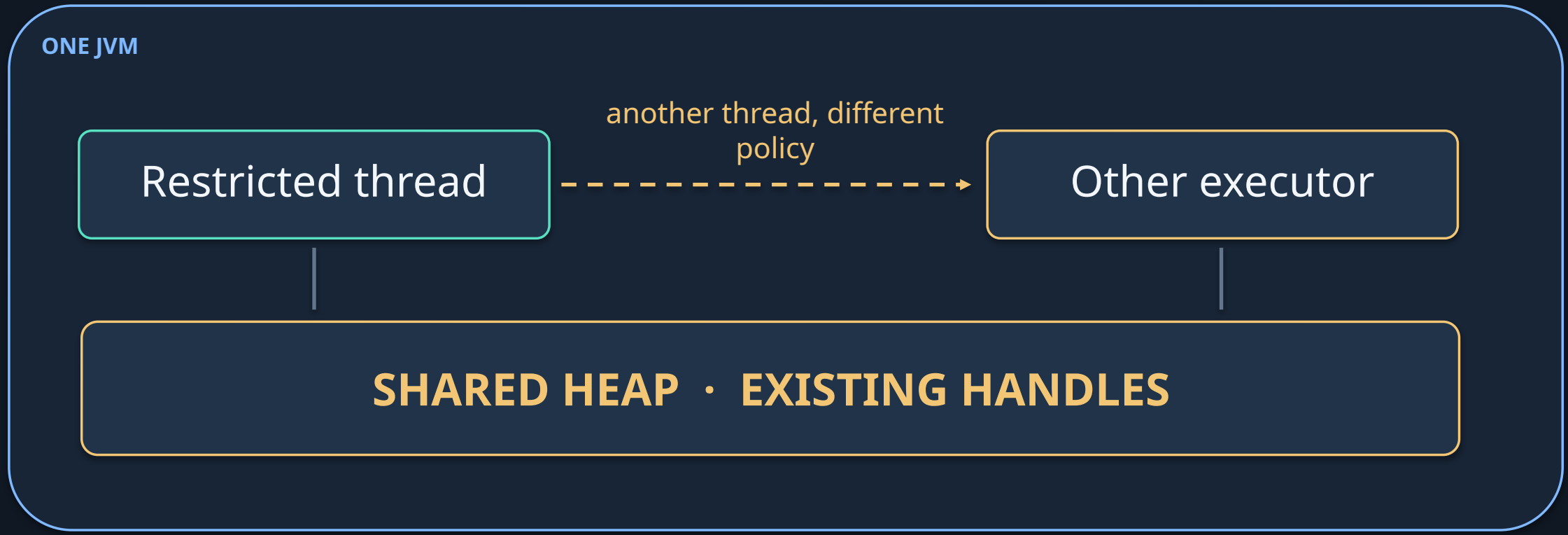
The method returns. The restrictions stay.



No “undo” in finally.

Reuse dedicated threads for work with the same policy.

One heap is still one heap.



Shared memory means this is not an arbitrary-code sandbox.

Choose the boundary that matches the threat.

WHOLE APPLICATION

Shared rules

Ban actions the whole application never needs.

Process-wide policy

SELECTED THREAD

Fewer permissions

Restrict direct OS actions of selected work.

Shared JVM memory remains

SEPARATE PROCESS

Separate memory

Separate address spaces. Harden communication.

Not necessarily a microservice

Security Manager: permanently disabled since JDK 24. This is a different model.